

Práctica 3 ITBD: Computación en la nube sin servidor

Kenjhy Bazan Turin

Nicolás Vives Vicente

Escuela Politécnica Superior

Máster Universitario en Ciencia de Datos

Infraestructuras y Tecnologías Big Data

Curso académico [2025-2026]



Universitat d'Alacant
Universidad de Alicante

Índice

1. Trabajo Local	3
1.1. Identificación de cámara pública	3
1.2. Captura local de frames	3
1.3. Detección de personas en frames	6
1.4. Visualización local	7
2. Externalización del Procesamiento bajo el Modelo Serverless en la nube	8
2.1. Página web estática en Azure	9
2.2. Carga de frames a la nube	9
2.3. Función serverless con blob trigger en Azure	11
2.4. Creación de un servicio de IA	12
2.5. Desarrollo de una función serverless	15
2.6. Conclusiones	19
3. Avanzado: Sistema inteligente que cuenta las personas y los vehículos que pasan por cada cámara en un periodo de tiempo	20

1. Trabajo Local

1.1. Identificación de cámara pública

Hemos escogido la webcam del siguiente link: <https://www.whatsupcams.com/en/webcams/italy/friuli-venezia-giulia/gorizia/gorizia-webcam-corso-verdi/>

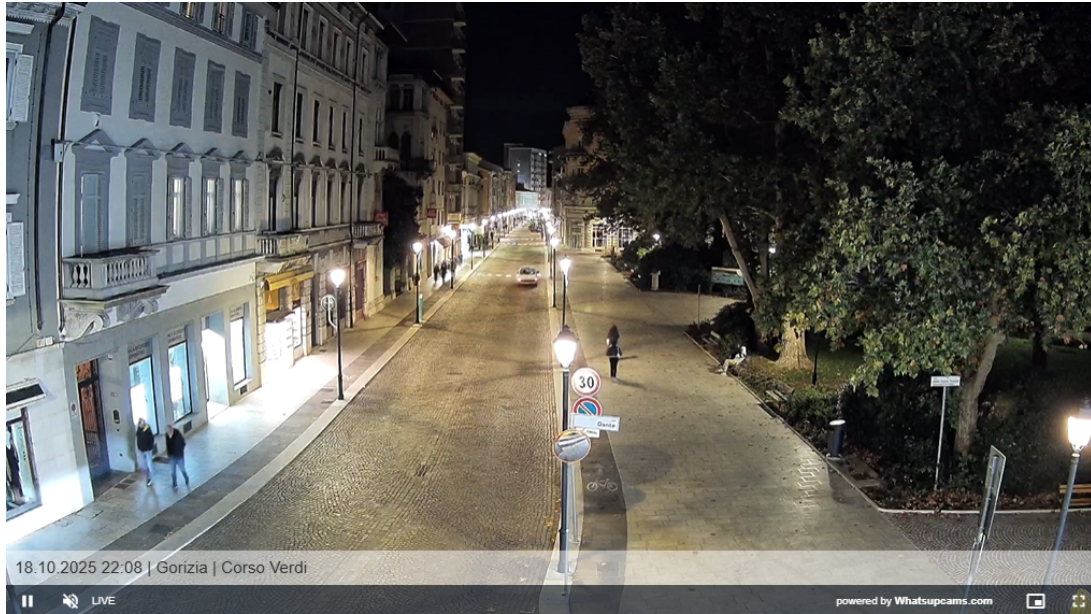


Figura 1: *Frame de la webcam elegida*

1.2. Captura local de frames

Incorporamos el código al completo, que nos permite capturar un frame de video de la webcam anterior con una frecuencia de 5 seg, realizar la identificación de las personas y el dibujo del rectángulo, tal y como se indica en la sección 1.3.

```
import cv2
import urllib.request
from ultralytics import YOLO
import time
import csv
import os
from datetime import datetime

csv_filename = "database.csv"
```

```
yolo_net = YOLO("yolov8n.pt") # load a pretrained model (recommended for training)

# URL of the MJPEG video stream
url = "https://cdn-007.whatsupcams.com/hls/go25_01.m3u8"

# Open the video stream
stream = urllib.request.urlopen(url)

# Create a VideoCapture object to read frames from the stream
cap = cv2.VideoCapture()

# Set the VideoCapture object to use the MJPEG stream
cap.open(url)

# Check if the stream is opened successfully
if not cap.isOpened():
    print("Error: Could not open the stream.")
    exit(1)

#CSV (En caso de que empecemos sin que exista el encabezado)
write_header = not os.path.exists(csv_filename)

#Contador para las imagenes
frame_counter = 0
max_images = 10

while True:
    # Read a frame from the stream
    ret, frame = cap.read()

    # Check if the frame was read successfully
    if not ret:
        print("Error: Could not read frame.")
        break

    # Perform person detection using YOLOv8
    #blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True, crop=False)
    detections = yolo_net.predict(source=frame)

    person_count = 0
    for detection in detections[0]:
        class_id = detection.bboxes.cls.item()
```

```
confidence = detection.bboxes.conf.item()

# Check if the detected object is a person (class_id 0)
if class_id == 0 and confidence > 0.5:
    # Extract bounding box coordinates
    center_x, center_y, width, height = detection.bboxes.xywh[0].tolist()
    x, y = (int(center_x - width / 2), int(center_y - height / 2))
    p1 = (x, y)
    p2=(int(x + width), int(y + height))

    # Draw a bounding box around the person
    cv2.rectangle(img=frame, pt1=p1, pt2=p2, color=(0, 255, 0), thickness=2)
    person_count = person_count + 1

# Get the current date and time with seconds precision
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# Display the frame with detected persons
#cv2.imshow("Person Detection", frame)
# cv2.imwrite('processed_frame.jpg', frame)
image_filename = None
if person_count > 0:
    image_filename = f"processed_frame_{frame_counter}.jpg"
    cv2.imwrite(image_filename, frame)
    frame_counter = (frame_counter + 1) % max_images

# Create or open the CSV file for writing
with open(csv_filename, mode='a', newline='') as csv_file:
    fieldnames = ["Timestamp", "Person_Count", "Image_Filename"]
    writer = csv.DictWriter(csv_file, fieldnames=fieldnames)

    if write_header:
        writer.writeheader()
        write_header = False

    # Write the timestamp and person count to the CSV file
    writer.writerow({"Timestamp": timestamp, "Person_Count": person_count,
                     "Image_Filename": image_filename if image_filename else None})

# Press 'q' to exit the loop
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
break

time.sleep(5)

# Release the VideoCapture and close the window
cap.release()
cv2.destroyAllWindows()
```

1.3. Detección de personas en frames

Como se comentaba en la sección 1.2 en el código mostrado, utilizando el modelo de IA Ultralytics YOLOv8 hemos sido capaces de identificar a personas en la captura tomada de la cámara.

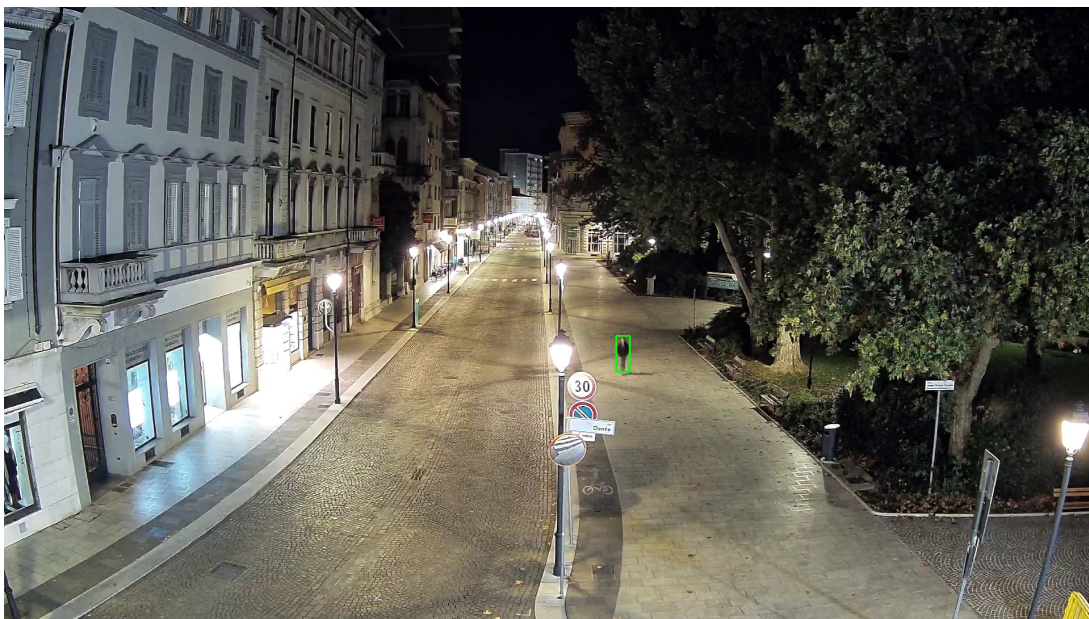


Figura 2: *Detección de personas*

Conseguimos también guardar en un csv la fecha y hora exacta junto con el número de personas detectadas en dicho instante. Nosotros hemos añadido que guarde únicamente las imágenes en las que ha detectado alguna persona y añada el nombre de la imagen al csv. También hemos limitado a 10 el número de imágenes ya que al ser en local no queríamos almacenar una gran cantidad, por lo que se van sobrescribiendo. Todo esto se puede ver en el código de la sección 1.2 y el resultado del csv en la Figura 3

```
database.csv
1193 2025-10-18 22:16:30,0,
1194 2025-10-18 22:16:31,0,
1195 2025-10-18 22:16:31,0,
1196 2025-10-18 22:16:32,0,
1197 2025-10-18 22:16:33,1,processed_frame_0.jpg
1198 2025-10-18 22:16:33,1,processed_frame_1.jpg
1199 2025-10-18 22:16:34,1,processed_frame_2.jpg
1200 2025-10-18 22:16:35,1,processed_frame_3.jpg
1201 2025-10-18 22:16:35,1,processed_frame_4.jpg
1202 2025-10-18 22:16:36,1,processed_frame_5.jpg
1203 2025-10-18 22:16:37,1,processed_frame_6.jpg
1204 2025-10-18 22:16:37,1,processed_frame_7.jpg
1205 2025-10-18 22:16:38,1,processed_frame_8.jpg
1206 2025-10-18 22:16:39,1,processed_frame_9.jpg
1207 2025-10-18 22:16:39,1,processed_frame_0.jpg
1208 2025-10-18 22:16:40,1,processed_frame_1.jpg
1209 2025-10-18 22:16:41,1,processed_frame_2.jpg
1210 2025-10-18 22:16:42,1,processed_frame_3.jpg
1211 2025-10-18 22:16:42,1,processed_frame_4.jpg
1212 2025-10-18 22:16:43,1,processed_frame_5.jpg
1213 2025-10-18 22:16:43,1,processed_frame_6.jpg
1214 2025-10-18 22:16:44,1,processed_frame_7.jpg
1215 2025-10-18 22:16:45,1,processed_frame_8.jpg
1216 2025-10-18 22:16:46,1,processed_frame_9.jpg
```

Figura 3: Archivo csv guardado tras la captura local

1.4. Visualización local

Modificando el archivo `index.html` hemos logrado crear una página web estática para visualizar los datos captados a través de una gráfica. Esta gráfica tiene la propiedad de mostrar la imagen del instante correspondiente a cada uno de los picos de la misma. Podemos observarlo en la Figura 4.

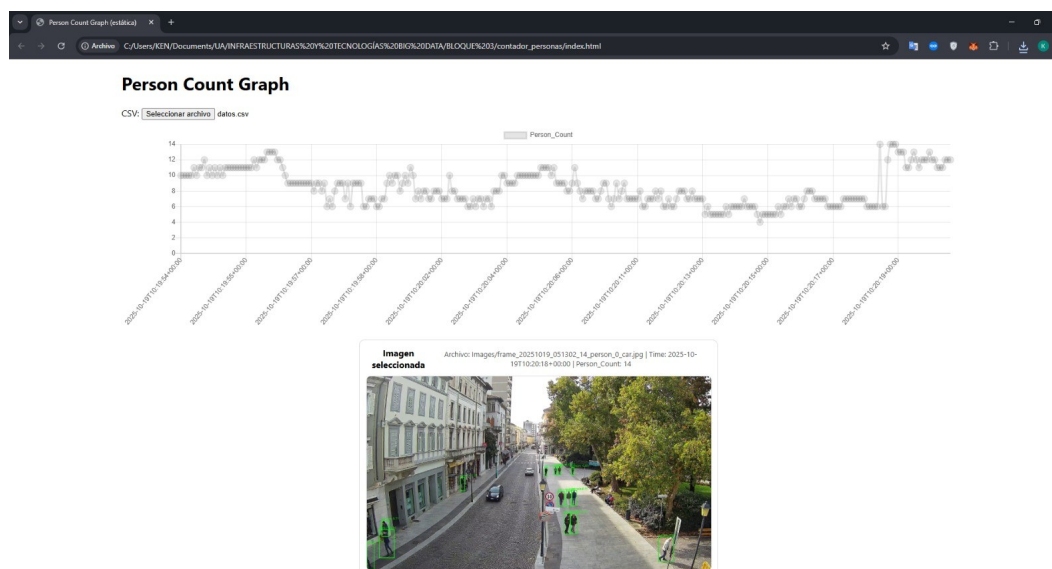


Figura 4: Página web estática

2. Externalización del Procesamiento bajo el Modelo Serverless en la nube

Para comenzar con el proceso en la nube de Azure debemos crear una nueva cuenta de almacenamiento junto con un grupo de recursos. En nuestro caso debido a problemas encontrados con Azure y con la suscripción **Azure for Students** hemos creado nuestra cuenta de almacenamiento en la región **Sweden Central** (Ver Figura 5). En el resto de opciones, tras hacer click en siguiente, nos aseguramos de permitir el acceso anónimo en contenedores individuales.

Datos básicos

Avanzado

Redes

Protección de datos

Cifrado

Etiquetas

Revisar y crear

Azure Storage es un servicio administrado por Microsoft que proporciona almacenamiento en la nube altamente disponible, seguro, duradero, escalable y redundante. Azure Storage incluye Azure Blob (objetos), Azure Data Lake Storage Gen2, Azure Files, Azure Queues y Azure Tables. El costo de una cuenta de Storage depende del uso y de las opciones que elija a continuación. [Más información sobre las cuentas de almacenamiento de Azure](#)

Detalles del proyecto

Seleccione la suscripción en la que se creará la nueva cuenta de almacenamiento. Elija un grupo de recursos nuevo o uno ya existente para organizar y administrar la cuenta de almacenamiento junto con otros recursos.

Suscripción *

Azure for Students

Grupo de recursos *

itbdnicolasvives2025

[Crear nuevo](#)

Detalles de la instancia

Nombre de la cuenta de almacenamiento *

itbdnicosweden

Región *

(Europe) Sweden Central

[Implementación en una zona extendida de Azure](#)

Tipo de almacenamiento preferido

Azure Blob Storage o Azure Data Lake Storage Gen2

!

 Esto nos ayuda a ofrecer orientación relevante. No limita su almacenamiento a este tipo de recurso. [Obtener más información](#)

Carga de trabajo principal

Seleccionar...

Elija la opción que más se asemeje a su carga de trabajo para obtener una configuración recomendada basada en los procedimientos recomendados. Puede editar esta configuración en cualquier momento. [Ver tabla de comparación](#)

Rendimiento *

☒ Estándar: Opción recomendada para la mayoría de los escenarios (cuenta de uso general v2)

☐ Prémium: Se recomienda para escenarios que requieren una latencia baja.

Redundancia *

Almacenamiento con redundancia local (LRS)

Anterior

Siguiente

Revisar y crear

Figura 5: Datos básicos de la cuenta de almacenamiento.

2.1. Página web estática en Azure

Comenzaremos creando la página web estática en nuestra cuenta de almacenamiento de Azure. Para ello, buscamos la opción **Sitio web estático** en la barra de búsqueda de la cuenta de almacenamiento.

Como se muestra en la Figura 6, hacemos clic en **Habilitado** y configuramos tanto el nombre del documento índice como la ruta de acceso del documento de error. En nuestro caso, hemos utilizado `index.html` y `not_found.html`, respectivamente.

A continuación, se crea automáticamente un contenedor en la cuenta de almacenamiento llamado `$web`. Dentro de este contenedor, subimos los archivos `index.html` y `not_found.html` que hemos heredado de la parte local.

De este modo, configuramos una página web estática que puede ser accesible públicamente a través de la siguiente URL:

`https://entagle3ua.z1.web.core.windows.net/`

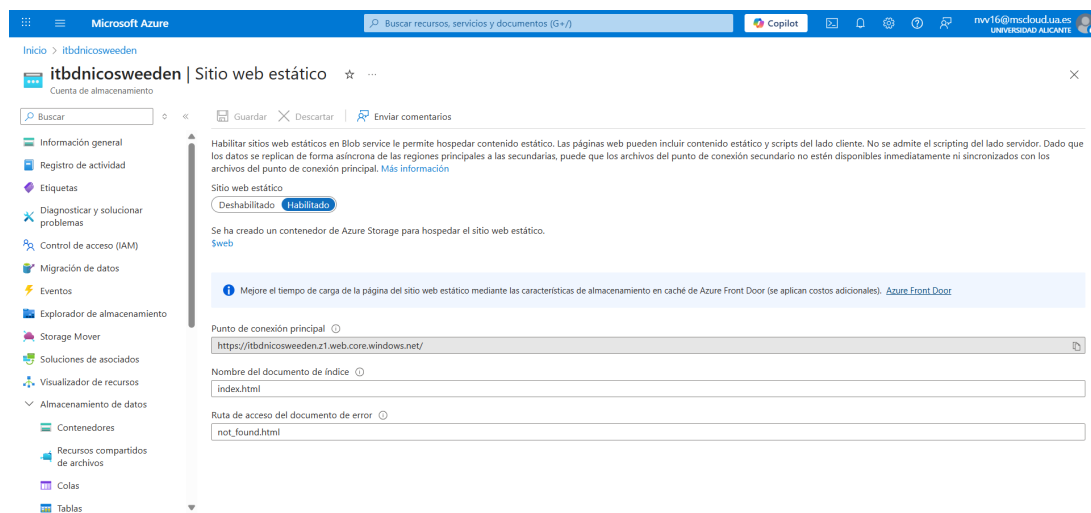


Figura 6: Creación de la página web estática en una cuenta del almacenamiento de Azure

2.2. Carga de frames a la nube

Utilizando el proceso descrito en la sección 1.2, hemos desarrollado el siguiente código que integra la captura de *frames* desde la webcam (sin procesar detectando personas en tiempo real) junto con el envío continuo de dichos *frames*, cada 0.5 segundos, a un contenedor llamado `contenedorframes` en nuestra cuenta de almacenamiento.

Para acceder a este contenedor, es fundamental obtener la cadena de conexión (*connection string*) de nuestra cuenta de almacenamiento, la cual configuramos como una variable de entorno. En el código, esta variable se referencia como AZURE_CONN_STR.

```
import os
import urllib.request
import cv2
from datetime import datetime
import time
from dotenv import load_dotenv
from azure.storage.blob import BlobServiceClient

# AZURE: conexión y contenedor

load_dotenv()

AZURE_CONN_STR = os.environ.get("AZURE_STORAGE_CONNECTION_STRING")

if not AZURE_CONN_STR:
    raise RuntimeError("No se encuentra AZURE_STORAGE_CONNECTION_STRING en variables de entorno.")

# Crear cliente del servicio Blob
blob_service_client = BlobServiceClient.from_connection_string(AZURE_CONN_STR)

container_name = "contenedorframe"

# Crear cliente para el contenedor
container_client = blob_service_client.get_container_client(container_name)

# CAPTURA DE LA IMAGEN

# URL of the MJPEG video stream
url = "https://cdn-007.whatsupcams.com/hls/it_gorizia02.m3u8"

# Open the video stream
stream = urllib.request.urlopen(url)

# Create a VideoCapture object to read frames from the stream
cap = cv2.VideoCapture()

# Set the VideoCapture object to use the MJPEG stream
cap.open(url)
```

```
# Check if the stream is opened successfully
if not cap.isOpened():
    print("Error: Could not open the stream.")
    exit(1)

while True:
    # Read a frame from the stream
    ret, frame = cap.read()

    # Check if the frame was read successfully
    if not ret:
        print("Error: Could not read frame.")
        break

    # Get the current date and time with seconds precision
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

    filename = f"frame_{timestamp}.jpg"
    cv2.imwrite(filename, frame)

    # Crear cliente para el blob (archivo)
    blob_client = container_client.get_blob_client(filename)

    # Leer y subir el archivo
    with open(filename, mode="rb") as data:
        blob_client.upload_blob(data, overwrite=True)

    print(f"Archivo {filename} subido correctamente al contenedor {container_name}.")

    time.sleep(0.5)

# Release the VideoCapture and close the window
cap.release()
cv2.destroyAllWindows()
```

2.3. Función serverless con blob trigger en Azure

En esta sección vamos a crear una función de tipo *Blob Trigger* para que se ejecute cada vez que una imagen sin procesar haya sido subida al contenedor **contenedorframes**.

Para ello, comenzamos creando una aplicación de funciones, debemos buscar entre los

servicios de Azure **Aplicación de funciones**, como se ve en la Figura 7.

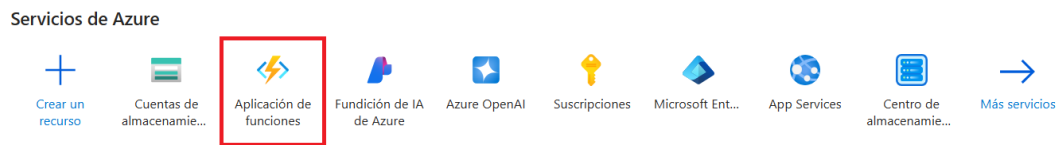


Figura 7: Algunos servicios de Azure.

Tras localizarlo, iniciamos la configuración seleccionando la opción **Crear**. En este punto, seleccionamos el modelo de hospedaje más adecuado según las necesidades del proyecto, en nuestro caso escogemos la opción **Consumo**.

A continuación, se definen los datos básicos, que configuramos como se ven en la Figura 8. Nuestra suscripción es **Azure for Students** y nuestro grupo de recursos previamente creado es **itbdnicolasvives2025**. Elegimos un nombre para la aplicación de funciones, en este caso elegimos **BlobTrigger** y seleccionamos el lenguaje Python que automáticamente bloquea el sistema operativo en Linux. Como región hemos escogido **Sweden Central** que es la que menos conflictos creaba con el resto del proyecto.

Hacemos click en Siguiente: Storage y de nuevo para evitar problemas, la cuenta de almacenamiento será una nueva que se creará automáticamente. El resto de opciones las dejamos por defecto. En la sección Redes nos aseguramos de **habilitar el acceso público** y en los siguientes de deshabilitar tanto **Application Insights** como la **Implementación Continua**. Por último vamos a **Revisar y Crear** y hacemos click en “Crear”.

Una vez creada la Aplicación de Funciones debemos crear la función de tipo *Blob Trigger*, sin embargo debido a la cantidad de problemas encontrados con Azure y la suscripción para estudiantes, vamos a crear esta función desde VSCode. Antes de desplegar nuestro proyecto de función en la aplicación de funciones de Azure debemos tener configurado correctamente el servicio de IA.

2.4. Creación de un servicio de IA

Como se comentaba en la sección 2.3 vamos a crear el servicio de IA de Azure que nos permita detectar a las personas que pasan por la webcam. Comenzamos buscando entre los servicios de Azure el servicio Fundición de IA de Azure que se muestra en la Figura 9.

Crear aplicación de funciones (Consumo) ...

Datos básicos Storage Redes Supervisión Implementación Autenticación Etiquetas Revisar y crear

Cree una aplicación de funciones, que permite agrupar funciones como una unidad lógica para facilitar la administración, la implementación y el uso compartido de los recursos. Fuctions permite ejecutar el código en un entorno sin servidor sin necesidad de crear primero una VM ni de publicar una aplicación web.

Detalles del proyecto

Seleccione una suscripción para administrar los recursos implementados y los costos. Use los grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción * ⓘ Azure for Students

Grupo de recursos * ⓘ itbdnicolasvives2025
[Crear nuevo](#)

Detalles de instancia

Nombre de la aplicación de funciones * BlobTrigger ✓
-gxbubjhcd5c0cub9.swedencentral-01.azurewebsites.net

☒ Nombre de host predeterminado único seguro (versión preliminar) activado. [Más información sobre esta actualización](#)

Sistema operativo * ☒ Linux (heredado) ☐ Windows

Para Linux, se recomienda usar Consumo flexible. [Obtenga más información.](#)

Pila del entorno en tiempo de ejecución Python

Versión * 3.12

Región * Sweden Central

[Revisar y crear](#) < Anterior Siguiente: Storage >

Figura 8: Relleno de datos básicos

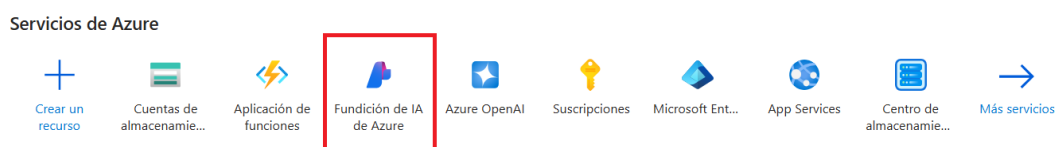


Figura 9: Servicio Fundición de IA de Azure.

Una vez localizado el servicio hacemos click en: Uso con AI Foundry > Fundición de IA de Azure. Como se muestra en la Figura 10.

Posteriormente seleccionamos “Crear”, y en la fase de datos básicos elegimos nuestra suscripción (**Azure for Students**), el grupo de recursos que hemos estado utilizando (**itbdnicolasvives2025**), le damos un nombre al servicio y escogemos la región, en nuestro caso de nuevo **Sweden Central** para evitar algunos problemas encontrados con otras regiones. (Ver Figura 11)

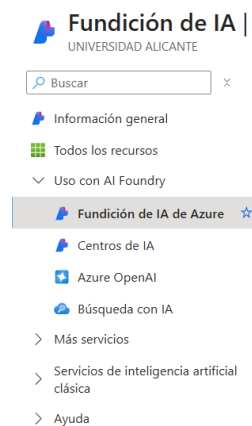


Figura 10: Menú de Fundación de IA.

Inicio > Fundación de IA | Fundación de IA de Azure >

Creación de un recurso de Fundación de IA de Azure

1 Datos básicos 2 Red 3 Identity 4 Cifrado 5 Etiquetas 6 Revisar y enviar

Diseñe, personalice y administre agentes y aplicaciones de inteligencia artificial a escala. [Obtener más información](#)

Detalles de la instancia

Seleccione la suscripción para administrar los costos y los recursos implementados. Use grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción * ⓘ Azure for Students

Grupo de recursos * ⓘ itbdnicolasvives2025 [Crear nuevo](#)

Nombre * ⓘ fundicionIA

Región ⓘ Sweden Central

[Ver todos los detalles de los precios](#)

Su primer proyecto

AI Foundry organiza los artefactos de desarrollo en proyectos: contenedores para administrar permisos, supervisión, costos y mucho más. Al crear un recurso, también se crea un proyecto. Este proyecto "predeterminado" tiene más funcionalidades que los proyectos que se crean más adelante. [Más información.](#)

Default project name * firstProject

Directiva de revisión de contenido

Fundación de IA de Azure proporciona acceso a los modelos de Azure OpenAI. Para detectar y mitigar el uso dañino de Azure OpenAI Service, Microsoft registra el contenido que envía a las API de generación de imágenes y finalizaciones, así como el contenido que devuelve. Si el contenido está marcado por los filtros del servicio, un empleado a tiempo completo de Microsoft puede revisarlo.

[Más información sobre cómo Microsoft procesa, usa y almacena tus datos](#)

< Anterior **Siguiente**

Figura 11: Datos básicos de la Fundación de IA.

Hacemos click en siguiente y en el apartado de Redes mantenemos la opción: “Todas las redes, incluso Internet, pueden acceder a este recurso”. Continuamos haciendo click en siguiente hasta crear el recurso.

Ahora vamos al recurso y en Administración de Recursos seleccionamos **Claves y punto de conexión**. Nos aseguramos aquí que nuestro recurso tenga los servicios de **Computer Vision**, que dependiendo de la región en la que lo hayamos creado esto puede variar. Copiamos tanto la clave 1 como el punto de conexión a los servicios de IA que son los que incorporamos al código que veremos en la sección 2.5. (Ver Figura 12)

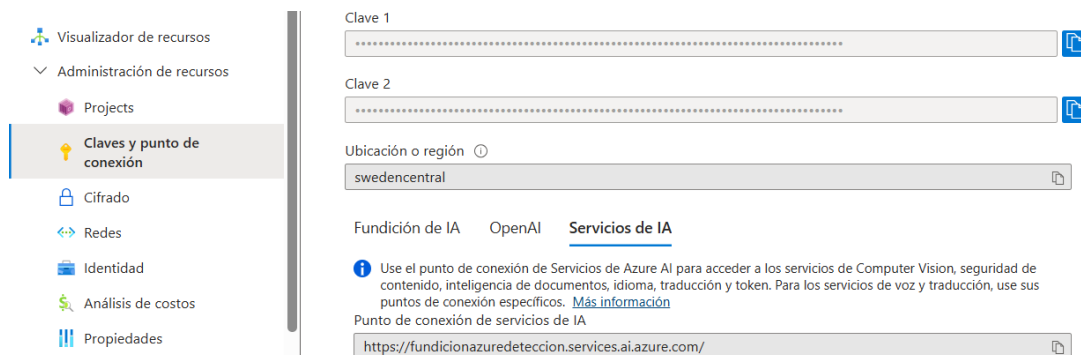


Figura 12: Claves y punto de conexión del recurso de IA.

2.5. Desarrollo de una función serverless

Una vez configurado este servicio, podremos editar el código de la función *trigger* como queremos.

Vamos a seguir los pasos descritos en <https://learn.microsoft.com/en-gb/azure/azure-functions/functions-develop-vs-code>. En primer lugar instalamos Azure Tools en VSCode e iniciamos sesión con nuestra cuenta de Azure. Una vez dentro hacemos **Ctrl + Shift + P** para abrir la paleta de comandos y buscamos **Azure Functions: Create New Project...**, seleccionamos una carpeta para alojar el proyecto, el lenguaje Python, el tipo de función *Blob Trigger*, el nombre de la función (en nuestro caso elegimos *blob-trigger* y el contenedor que activa el trigger (**contenedorframes** en nuestro caso). Una vez creado el proyecto encontraremos en la carpeta seleccionada los archivos que vemos en la Figura 13.

Ahora debemos modificar algunos de estos archivos que se han creado, veamos qué contienen los archivos que hemos modificado. El script `function_app.py` contiene el siguiente código:

```
import os
import logging
import numpy as np
```

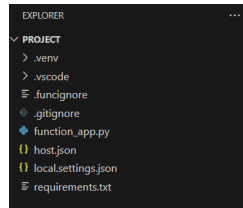


Figura 13: Proyecto de función Blob Trigger.

```
import cv2
import azure.functions as func
from azure.storage.blob import BlobServiceClient
from azure.core.credentials import AzureKeyCredential
from azure.ai.vision.imageanalysis import ImageAnalysisClient
from azure.ai.vision.imageanalysis.models import VisualFeatures

app = func.FunctionApp()

@app.blob_trigger(
    arg_name="myblob",
    path="contenedorframe/{name}", # <<< tu contenedor de entrada + placeholder
    connection="itbdnicosweeden_STORAGE" # <<< nombre de la app setting con la
        connection string
)
def blob_trigger(myblob: func.InputStream):
    logging.info(f"[BlobTrigger] name={myblob.name} size={myblob.length} bytes")

    # === App Settings (en Configuration de la Function App) ===

    endpoint = os.environ["VISION_ENDPOINT"]
    vision_key = os.environ["VISION_KEY"]
    storage_conn = os.environ["itbdnicosweeden_STORAGE"]
    out_container = os.environ.get("OUTPUT_CONTAINER", "$web")

    # 1) bytes de la imagen desde el trigger
    content = myblob.read()

    # 2) llamar a Image Analysis (PEOPLE)
    vision = ImageAnalysisClient(endpoint=endpoint,
        credential=AzureKeyCredential(vision_key))
    result = vision.analyze(image_data=content,
        visual_features=[VisualFeatures.PEOPLE], language="en")

    # 3) decodificar y dibujar cuadros
```

```

np_bytes = np.frombuffer(content, dtype=np.uint8)
img = cv2.imdecode(np_bytes, cv2.IMREAD_COLOR)
if img is None:
    logging.error("No se pudo decodificar la imagen")

person_count = 0
if getattr(result, "people", None):
    for p in result.people.list:
        conf = float(getattr(p, "confidence", 0.0) or 0.0)
        if conf < 0.5:
            continue
        bb = getattr(p, "bounding_box", None)
        if bb is None:
            continue
        x = int(getattr(bb, "x", getattr(bb, "left", 0)))
        y = int(getattr(bb, "y", getattr(bb, "top", 0)))
        w = int(getattr(bb, "w", getattr(bb, "width", 0)))
        h = int(getattr(bb, "h", getattr(bb, "height", 0)))
        cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0), 2)
        cv2.putText(img, f"person {conf:.2f}", (x, max(0, y-5)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1, cv2.LINE_AA)
        person_count += 1

ok, out_buf = cv2.imencode(".jpg", img, [int(cv2.IMWRITE_JPEG_QUALITY), 90])
if not ok:
    logging.error("No se pudo codificar la imagen anotada")
out_bytes = out_buf.tobytes()

# 4) subir a contenedor de salida
blob_service = BlobServiceClient.from_connection_string(storage_conn)
out_client = blob_service.get_container_client(out_container)

try:
    if out_container != '$web':
        out_client.create_container()
    print(f"Contenedor '{out_container}' confirmado o creado")
except Exception as e:
    print(f"Error creando contenedor: {e}")

try:
    out_name = f'processed_{myblob.name}.jpg'
    out_client.upload_blob(name=out_name, data=out_bytes, overwrite=True)
    print(f"Imagen subida a {out_container}/{out_name}")

```

```
except Exception as e:
    print(f"Error subiendo imagen: {e}")
```

Este código configura una función de tipo *Blob Trigger* que se activa cuando se sube un *blob* al contenedor **contenedorframes**, la función se conecta con el servicio de IA de Azure configurado en la sección 2.4 detectando a las personas del frame. Después dibuja el cuadro alrededor de la persona y sube la imagen procesada al contenedor **\$web**.

Al archivo `local.settings.json` generado en el proyecto le debemos añadir las variables de entorno siguientes, de las cuales no mostramos aquí su valor.

```
"VISION_ENDPOINT": -
"VISION_KEY": -
"itbdnicosweeden_STORAGE": -
"OUTPUT_CONTAINER": "$web"
```

El último archivo que debemos modificar es `requirements.txt`, que contiene los paquetes necesarios para la ejecución de la función. En nuestro caso:

```
azure-functions
azure-storage-blob>=12.21.0
azure-ai-vision-imageanalysis>=1.0.0b3
opencv-python-headless
numpy
```

Una vez configurados estos archivos desplegamos la carpeta del proyecto como una función en la aplicación de funciones de Azure.

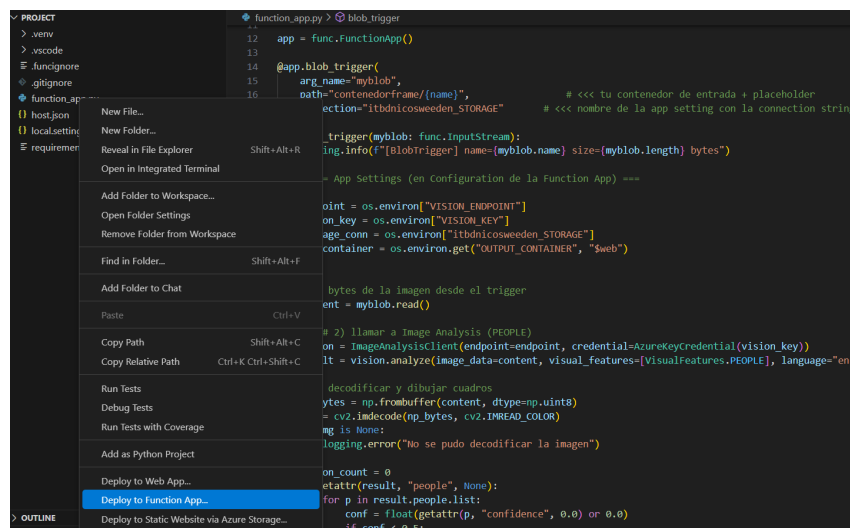


Figura 14: Desplegar en la aplicación de funciones.

Comprobamos en Azure que efectivamente la función está habilitada (Figura 15)

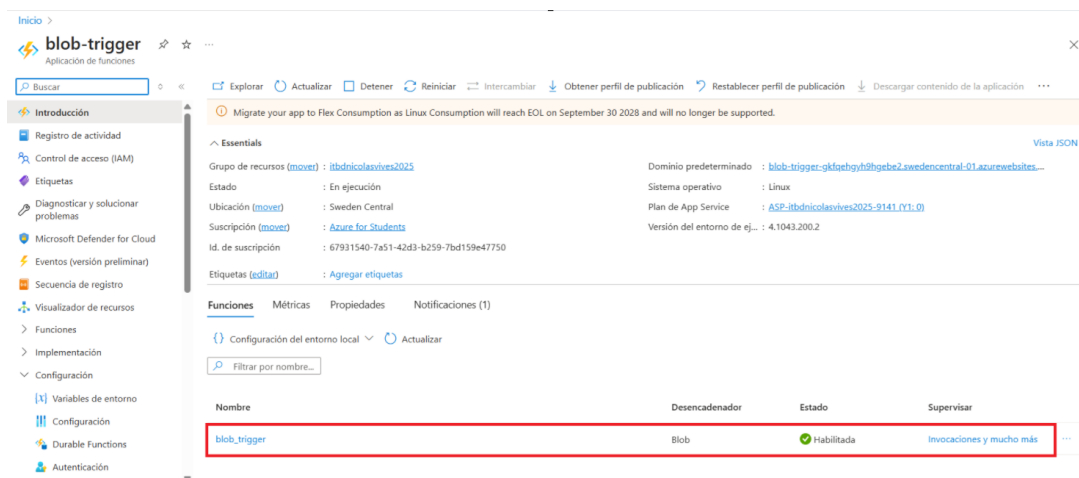


Figura 15: Función desplegada en la aplicación de funciones.

Una vez la función está operativa si ejecutamos nuestro código que envía las imágenes sin procesar a nuestro contenedor **contenedorframe**, cada vez que se sube una imagen, el trigger se activa, se procesa y se envía al contenedor **\$web**, tal y como se ve en la Figura 16.

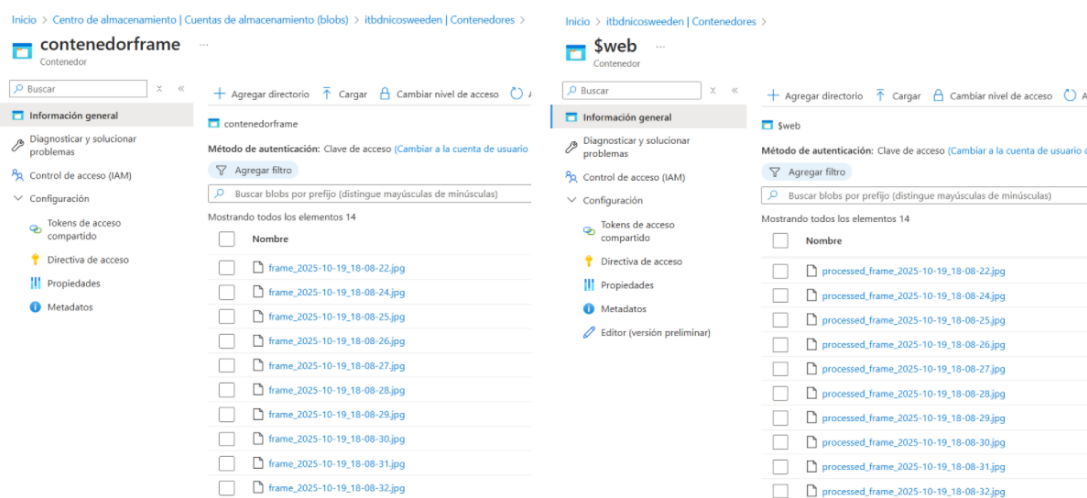


Figura 16: Imágenes sin procesar y procesadas subidas a ambos contenedores.

2.6. Conclusiones

Este proyecto nos ha permitido comprobar el potencial de Azure Functions para automatizar procesos en la nube, aunque también nos ha hecho ver sus complicaciones

prácticas. La configuración en Azure unida a las limitaciones de la suscripción **Azure for Students** nos ha generado bastantes problemas técnicos que nos han obligado a invertir más tiempo del previsto. Aun así, hemos aprendido mucho sobre despliegue en la nube y sobre cómo funcionan los triggers y permisos dentro del ecosistema Azure. Después de esta experiencia, pensamos que otros servicios cloud, como AWS o Google Cloud Platform (GCP), podrían ofrecer un entorno más sencillo y ágil, sobre todo para proyectos académicos.

3. Avanzado: Sistema inteligente que cuenta las personas y los vehículos que pasan por cada cámara en un periodo de tiempo

Para realizar esta parte extra hemos desarrollado el siguiente código que recoge las imágenes subidas al contenedor **contenedorframe**, las procesa detectando el número de personas y de coches y siempre y cuando en la imagen haya alguna persona o algún coche las guarda como “frame_AAAAMMDD_HHMMSS_numpersonas_person_numcoches_car” para después subirlas procesadas y con este nombre al contenedor **\$web**.

También se guardan en un csv por cada imagen procesada **Timestamp**, **Person_Count**, **Car_Count**, **Image_Filename** como en la Figura 17.

```

1  Timestamp,Person_Count,Car_Count,Image_Filename
2  2025-10-18T20:02:37+00:00,1,0,frame_20251018_150236_1_person_0_car.jpg
3  2025-10-18T20:02:39+00:00,1,0,frame_20251018_150238_1_person_0_car.jpg
4  2025-10-18T20:02:45+00:00,1,0,frame_20251018_150245_1_person_0_car.jpg
5  2025-10-18T20:02:47+00:00,1,0,frame_20251018_150246_1_person_0_car.jpg
6  2025-10-18T20:02:49+00:00,1,0,frame_20251018_150249_1_person_0_car.jpg
7  2025-10-18T20:02:50+00:00,1,0,frame_20251018_150250_1_person_0_car.jpg
8  2025-10-18T20:02:56+00:00,1,0,frame_20251018_150256_1_person_0_car.jpg
9  2025-10-18T20:02:57+00:00,1,0,frame_20251018_150257_1_person_0_car.jpg
10 2025-10-18T20:02:59+00:00,1,0,frame_20251018_150258_1_person_0_car.jpg
11 2025-10-18T20:03:00+00:00,1,0,frame_20251018_150300_1_person_0_car.jpg
12 2025-10-18T20:03:01+00:00,1,0,frame_20251018_150301_1_person_0_car.jpg
13 2025-10-18T20:03:02+00:00,1,0,frame_20251018_150302_1_person_0_car.jpg
14 2025-10-18T20:04:27+00:00,1,0,frame_20251018_150426_1_person_0_car.jpg
15 2025-10-18T20:04:28+00:00,1,0,frame_20251018_150428_1_person_0_car.jpg
16 2025-10-18T20:04:31+00:00,1,0,frame_20251018_150431_1_person_0_car.jpg
17 2025-10-18T20:04:45+00:00,1,0,frame_20251018_150445_1_person_0_car.jpg
18 2025-10-18T20:06:11+00:00,2,0,frame_20251018_150611_2_person_0_car.jpg
19 2025-10-18T20:06:12+00:00,2,0,frame_20251018_150612_2_person_0_car.jpg

```

Figura 17: Muestra del formato de guardado en el csv.

```
import time
import io
import csv
from datetime import datetime, timezone

import numpy as np
import cv2
from azure.storage.blob import BlobServiceClient, ContentSettings
from azure.core.credentials import AzureKeyCredential
from azure.ai.vision.imageanalysis import ImageAnalysisClient
from azure.ai.vision.imageanalysis.models import VisualFeatures

# =====
# CONFIGURACIÓN LOCAL
# =====

endpoint = os.environ["VISION_ENDPOINT"]
vision_key = os.environ["VISION_KEY"]
storage_conn = os.environ["itbdnicosweedden_STORAGE"]

in_container = "contenedorframe" # contenedor de entrada
out_container = "$web" # contenedor de salida

# =====
# FUNCIONES AUXILIARES
# =====

def is_image_blob(name: str) -> bool:
    return name.lower().endswith((".jpg", ".jpeg", ".png"))

def draw_people_boxes(img_bgr, result, conf_thresh=0.5):
    """Dibuja cuadros verdes sobre las personas detectadas."""
    person_count = 0
    if getattr(result, "people", None):
        for p in result.people.list:
            conf = float(getattr(p, "confidence", 0.0) or 0.0)
            if conf < conf_thresh:
                continue
            bb = getattr(p, "bounding_box", None)
            if bb is None:
                continue
            x = int(getattr(bb, "x", getattr(bb, "left", 0)))
```

```

y = int(getattr(bb, "y", getattr(bb, "top", 0)))
w = int(getattr(bb, "w", getattr(bb, "width", 0)))
h = int(getattr(bb, "h", getattr(bb, "height", 0)))
cv2.rectangle(img_bgr, (x, y), (x + w, y + h), (0, 255, 0), 2) # verde
cv2.putText(img_bgr, f"person {conf:.2f}",
            (x, max(0, y - 5)), cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (0, 255, 0), 1, cv2.LINE_AA)
person_count += 1
return person_count, img_bgr

def get_object_labels(obj):
    """Obtiene etiquetas de un objeto detectado de forma robusta."""
    labels = []
    tags = getattr(obj, "tags", None)
    if tags:
        for t in tags:
            name = getattr(t, "name", None)
            if name:
                labels.append(str(name).lower())
    name_attr = getattr(obj, "name", None)
    if name_attr:
        labels.append(str(name_attr).lower())
    return list(dict.fromkeys(labels)) # únicos, orden estable

def draw_car_boxes(img_bgr, result, conf_thresh=0.5):
    """Detecta objetos tipo coche/vehículo y dibuja cuadros azules."""
    car_like = {"car", "automobile", "vehicle", "truck", "bus", "van", "taxi",
                "pickup", "suv"}
    car_count = 0
    if getattr(result, "objects", None):
        for o in result.objects.list:
            conf = float(getattr(o, "confidence", 0.0) or 0.0)
            if conf < conf_thresh:
                continue

            labels = set(get_object_labels(o))
            if not labels & car_like:
                continue # no es un coche/camión/etc.

            bb = getattr(o, "bounding_box", None)
            if bb is None:
                continue
            x = int(getattr(bb, "x", getattr(bb, "left", 0)))

```

```

y = int(getattr(bb, "y", getattr(bb, "top", 0)))
w = int(getattr(bb, "w", getattr(bb, "width", 0)))
h = int(getattr(bb, "h", getattr(bb, "height", 0)))

cv2.rectangle(img_bgr, (x, y), (x + w, y + h), (255, 0, 0), 2) # azul
lbl = next(iter(labels & car_like)) if (labels & car_like) else "car"
cv2.putText(img_bgr, f"{lbl} {conf:.2f}",
            (x, max(0, y - 5)), cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (255, 0, 0), 1, cv2.LINE_AA)

car_count += 1
return car_count, img_bgr

# =====
# PROCESO PRINCIPAL (LOTE)
# =====

def main():
    # Clientes de Blob y Vision
    blob_service = BlobServiceClient.from_connection_string(storage_conn)
    in_client = blob_service.get_container_client(in_container)
    out_client = blob_service.get_container_client(out_container)

    try:
        if out_container != "$web":
            out_client.create_container()
    except Exception:
        pass

    vision = ImageAnalysisClient(endpoint=endpoint,
                                  credential=AzureKeyCredential(vision_key))

    # Listar y ordenar imágenes del contenedor de entrada
    blobs = [b for b in in_client.list_blobs() if is_image_blob(b.name)]
    if not blobs:
        print(f" No hay imágenes en '{in_container}'")
        return

    blobs.sort(key=lambda b: b.last_modified) # más antiguo -> reciente

    total = len(blobs)
    saved = 0
    skipped_empty = 0
    skipped_already = 0

```

```
# --- acumulador CSV (solo imágenes procesadas/guardadas) ---
csv_rows = [] # [timestamp_iso, person_count, car_count, out_name]

for idx, b in enumerate(blobs, 1):
    name = b.name

    # Omitir las que ya parecen procesadas (evita reprocesar)
    if "_file_" in name:
        skipped_already += 1
        continue

    print(f"[{idx}/{total}] Descargando: {name}")
    img_bytes = in_client.get_blob_client(name).download_blob().readall()

    # Analizar (personas + objetos)
    try:
        result = vision.analyze(
            image_data=img_bytes,
            visual_features=[VisualFeatures.PEOPLE, VisualFeatures.OBJECTS],
            language="en"
        )
    except Exception as e:
        print(f" Error Vision en '{name}': {e}")
        continue

    # Decodificar y anotar
    np_bytes = np.frombuffer(img_bytes, dtype=np.uint8)
    img = cv2.imdecode(np_bytes, cv2.IMREAD_COLOR)
    if img is None:
        print(f" No se pudo decodificar '{name}'")
        continue

    person_count, img_annot = draw_people_boxes(img, result, conf_thresh=0.5)
    car_count, img_annot = draw_car_boxes(img_annot, result, conf_thresh=0.5)

    if person_count == 0 and car_count == 0:
        skipped_empty += 1
        print(f" Sin hallazgos en '{name}', no se guarda.")
        continue

    # Codificar y subir solo si hay hallazgos
    ok, out_buf = cv2.imencode(".jpg", img_annot, [int(cv2.IMWRITE_JPEG_QUALITY),
```

```
90])
if not ok:
    print(f" No se pudo codificar la imagen anotada '{name}')"
    continue
out_bytes = out_buf.tobytes()

base_name, ext = name.rsplit(".", 1)
out_name = f"{base_name}_{person_count}_person_{car_count}_car.{ext}"

out_client.get_blob_client(out_name).upload_blob(
    data=out_bytes,
    overwrite=True,
    content_settings=ContentSettings(content_type="image/jpeg")
)

saved += 1
print(f" Guardado en {out_container}/{out_name}")

# --- fila CSV SOLO para imágenes guardadas ---
ts_iso = b.last_modified.astimezone(timezone.utc).isoformat()
csv_rows.append([ts_iso, person_count, car_count, out_name])

# (Opcional) si quieres ser amable con la API:
time.sleep(0.15)

# --- generar y subir CSV (solo si hubo guardados) ---
if csv_rows:
    csv_name =
        f"detections_{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S')}.csv"
    buf = io.StringIO()
    writer = csv.writer(buf)
    writer.writerow(["Timestamp", "Person_Count", "Car_Count", "Image_Filename"])
    writer.writerows(csv_rows)
    data = buf.getvalue().encode("utf-8")

    out_client.get_blob_client(csv_name).upload_blob(
        data=data,
        overwrite=True,
        content_settings=ContentSettings(content_type="text/csv; charset=utf-8")
    )
    print(f"\n CSV subido: {out_container}/{csv_name}")
else:
    print("\n No se generó CSV (no hubo imágenes guardadas).")
```

```

print("\n==== RESUMEN LOTE =====")
print(f"Total en contenedor: {total}")
print(f"Guardados (con hallazgos): {saved}")
print(f"Omitidos (sin hallazgos): {skipped_empty}")
print(f"Omitidos (ya procesados _file_): {skipped_already}")

# =====
# EJECUCIÓN LOCAL
# =====

if __name__ == "__main__":
    main()

```

Hemos modificado también el archivo `index.html` de la página web estática para que la gráfica muestre en verde el número de personas y en azul el número de vehículos que hay en la imagen en función del tiempo. (Ver Figura 18)

Para la detección continua de personas la función debería tener en cuenta que tiempos muy próximos con cuadros de similar tamaño implican que se estaría detectando la misma persona o vehículo, nosotros lo hemos implementado en el `index.html` para que lo tenga en cuenta en la gráfica.

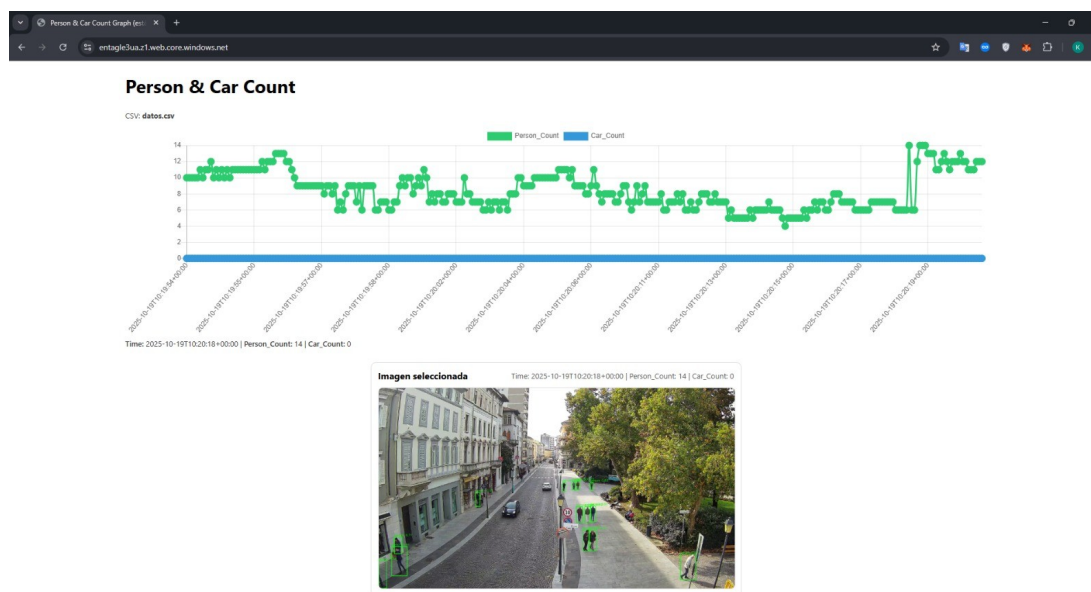


Figura 18: Web estática que muestra el número de personas y de coches en función del tiempo.

Como se puede observar en la Figura 18, la gráfica que mide los vehículos permanece constante, esto es debido a las limitaciones del servicio de IA que no consigue detectarlos y ofrece recursos como tales a cierto coste.